

SPA con React

2. Componentes



Universidad
Rey Juan Carlos

Micael Gallego

Correo: micael.gallego@urjc.es

Twitter: [@micael_gallego](https://twitter.com/micael_gallego)



Componentes

- **Componentes en React**
 - Un componente es una **nueva etiqueta HTML** con una **vista** y una **lógica** definidas por el desarrollador
 - La **vista** es una plantilla (*template*) que genera HTML basado en valores de variables
 - La **lógica** es el código TypeScript que prepara los datos de la vista y gestiona sus eventos

- **Componentes en React**
- Se usa una sintaxis especial (JSX/TSX) que permite especificar HTML en código JavaScript/TypeScript

home.tsx

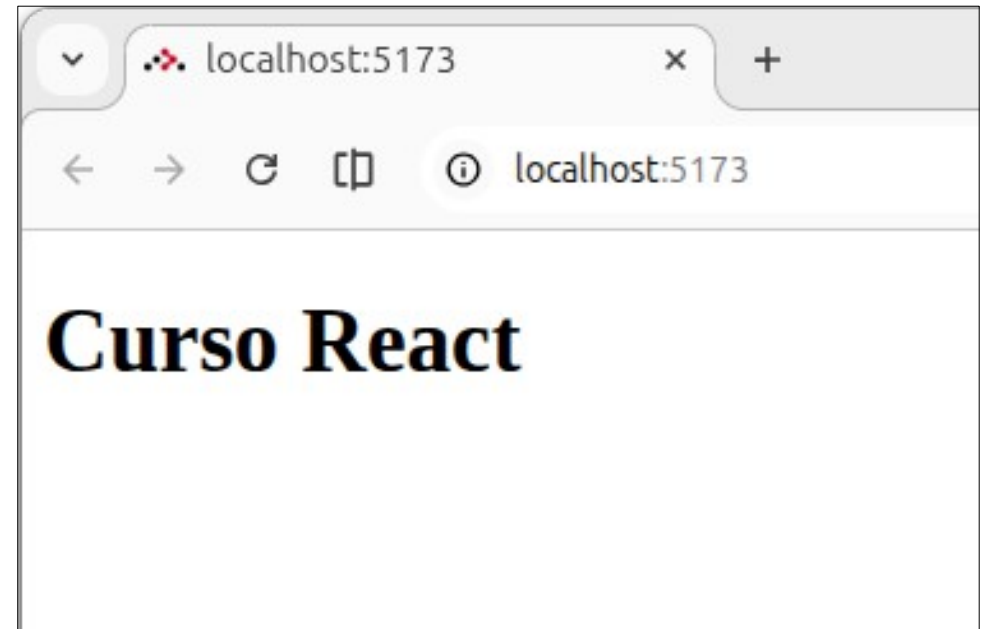
```
export default function Home() {  
  return (  
    <h1>  
      Curso React  
    </h1>  
  );  
}
```

Componentes

examo_minimal

home.tsx

```
export default function Home() {  
  return (  
    <h1>  
      Curso React  
    </h1>  
  );  
}
```

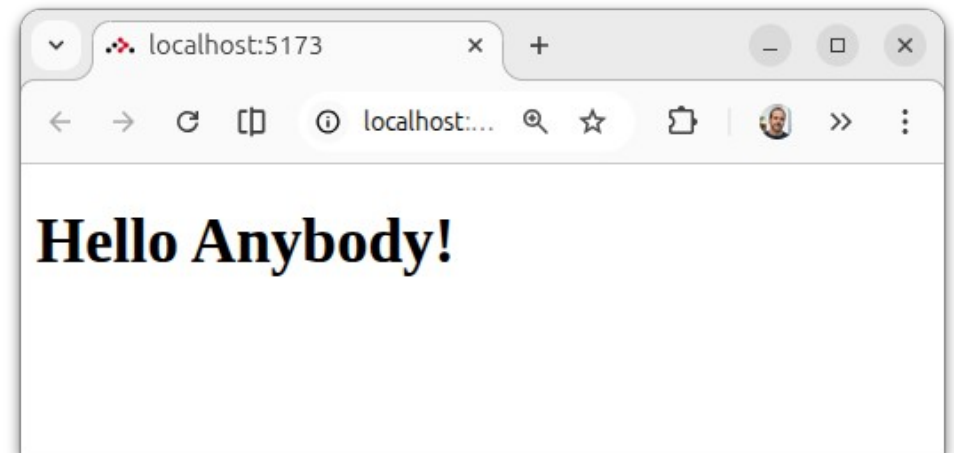


Visualizar datos dinámicos

La vista del componente (**HTML**) se genera en función de las variables de la función

home.tsx

```
export default function Home() {  
  const name = "Anybody";  
  
  return (  
    <h1>  
      Hello {name}!  
    </h1>  
  );  
}
```

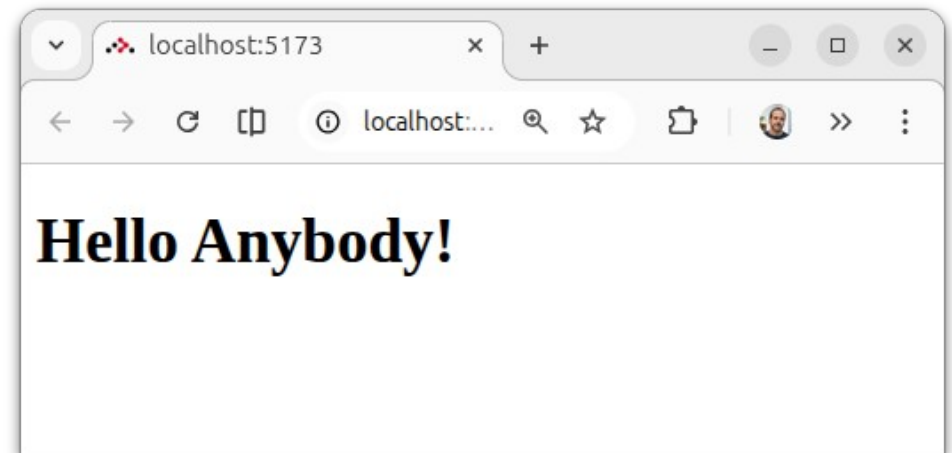
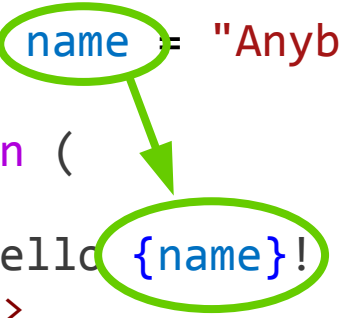


Visualizar datos dinámicos

La vista del componente (**HTML**) se genera en función de las variables de la función

home.tsx

```
export default function Home() {  
  const name = "Anybody";  
  return (  
    <h1>  
      Hello {name}!  
    </h1>  
  );  
}
```



Plantillas

- HTML en variables

home.tsx

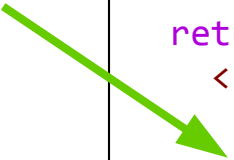
```
export default function Home() {  
  const logged = true;  
  let message;  
  
  if(logged) {  
    message = <>  
      <h1>Welcome back!</h1>  
      <p>You have successfully logged in.</p>  
    </>;  
  } else {  
    message = <p>Please log in</p>;  
  }  
  
  return (  
    <>  
      <div>Company Logo</div>  
      {message}  
      <div>Footer</div>  
    </>  
  );  
}
```

Se asigna HTML
directamente a
la variable

Cuando hay varios
elementos, se agrupan
en <> </>

Se incrusta el
contenido de la
variable

- Renderizado condicional
- if/else más compacto que con variables



```
export default function Home() {  
  const logged = true;  
  return (  
    <>  
    <div>Company Logo</div>  
    {logged ?  
      <>  
        <h1>Welcome back!</h1>  
        <p>You have successfully logged in.</p>  
      </>  
      :  
      <p>Please log in</p>  
    }  
    <div>Footer</div>  
  </>  
  );  
}
```

- Renderizado condicional
- Sólo if (sin else) más compacto todavía

home.tsx

```
export default function Home() {  
  const visible = true;  
  
  return (  
    <>  
      {visible && <p>Text</p>}  
    </>  
  );  
}
```

- Renderizado de arrays

home.tsx

```
export default function Home() {  
  
  const elems = [  
    { id: 1, desc: 'Elem1', visible: true },  
    { id: 2, desc: 'Elem2', visible: true },  
    { id: 3, desc: 'Elem3', visible: false }  
  ]  
  
  return (  
    <ul>  
      { elems.map(elem => <li key={elem.id}>{elem.desc}</li>) }  
    </ul>  
  );  
}
```

```
<ul>  
  <li>Elem1</li>  
  <li>Elem2</li>  
  <li>Elem3</li>  
</ul>
```

• Renderizado de arrays

home.tsx

```
export default function Home() {  
  const elems = [  
    { id: 1, desc: 'Elem1', visible: true },  
    { id: 2, desc: 'Elem2', visible: true },  
    { id: 3, desc: 'Elem3', visible: false }  
  ]  
  
  return (  
    <ul>  
      { elems.map(elem => <li key={elem.id}>{elem.desc}</li>) }  
    </ul>  
  );  
}
```

Se recomienda poner una **key** diferente a cada elemento por eficiencia si se eliminan o añaden elementos al array

Se puede mostrar un array de elementos HTML

Se suele usar la operación `.map()` para convertir cada elemento en su HTML

```
<ul>  
  <li>Elem1</li>  
  <li>Elem2</li>  
  <li>Elem3</li>  
</ul>
```

- Renderizado de arrays

home.tsx

```
export default function Home() {  
  const elems = [  
    { desc: 'Elem1', visible: true },  
    { desc: 'Elem2', visible: true },  
    { desc: 'Elem3', visible: false }  
  ]  
  
  return (  
    <ul>  
      { elems.map((elem, index) => <li key={index}>{elem.desc}</li>) }  
    </ul>  
  );  
}
```

Si los objetos no tienen key, se puede usar su posición en el array (**index**) (pero es más ineficiente)

```
<ul>  
  <li>Elem1</li>  
  <li>Elem2</li>  
  <li>Elem3</li>  
</ul>
```

- Renderizado de arrays y condicional

home.tsx

```
export default function Home() {  
  
  const elems = [  
    { id: 1, desc: 'Elem1', visible: true },  
    { id: 2, desc: 'Elem2', visible: true },  
    { id: 3, desc: 'Elem3', visible: false }  
  ]  
  
  return (  
    <ul>  
      { elems.map(elem =>  
        elem.visible && <li key={elem.id}>{elem.desc}</li>  
      )}  
    </ul>  
  );  
}
```

```
<ul>  
  <li>Elem1</li>  
  <li>Elem2</li>  
</ul>
```

- Renderizado de arrays y condicional

home.tsx

```
export default function Home() {  
  
  const elems = [  
    { id: 1, desc: 'Elem1', visible: true },  
    { id: 2, desc: 'Elem2', visible: true },  
    { id: 3, desc: 'Elem3', visible: false }  
  ]  
  
  return (  
    <ul>  
      { elems.map(elem =>  
        elem.visible && <li key={elem.id}>{elem.desc}</li>  
      )}  
    </ul>  
  );  
}
```

Si se devuelve **false** el elemento
no se muestra en el HTML

```
<ul>  
  <li>Elem1</li>  
  <li>Elem2</li>  
</ul>
```


Eventos

- Ejecución de lógica
- Se puede ejecutar una función ante un evento

home.tsx

```
export default function Home() {  
  function handleClick() {  
    alert("Hello John");  
  }  
  return (  
    <>  
      <button onClick={ handleClick }>  
        Hello John  
      </button >  
    </>  
  );  
}
```

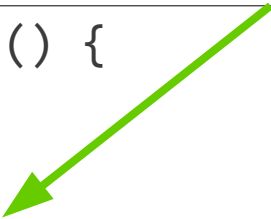
La función que se ejecutará
(sin paréntesis)



- Ejecución de lógica
- Se puede ejecutar una *arrow function* ante un evento

home.tsx

```
export default function Home() {  
  return (  
    <>  
      <button onClick={() => alert("Hello John")}>  
        Hello John  
      </button >  
    </>  
  );  
}
```



Estado

- No se pueden cambiar variables ante eventos

home.tsx

```
export default function Home() {  
  let name = "Anybody";  
  function handleClick() {  
    name = "John";  
  }  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
      <button onClick={handleClick}>Hello John</button>  
    </>  
  );  
}
```

No hay error, pero el
componente no se
actualiza

- Para que un componente se actualice al actualizar una variable, se debe encapsular en un **estado** (*state*)
- Para modificar el estado se debe usar una **función** (no se puede asignar la variable)
- La **variable** y la **función** que permite su actualización se consiguen invocando un *hook*

home.tsx

```
export default function Home() {  
  let name = "Anybody";  
  function handleClick() {  
    name = "John";  
  }  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```

No funciona!

home.tsx

```
import { useState } from "react";  
export default function Home() {  
  const [name, setName] =  
    useState("Anybody");  
  function handleClick() {  
    setName("John");  
  }  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```

Estado

useState(...) permite obtener la variable y la función que cambia esa variable

home.tsx

```
export default function Home() {  
  let name = "Anybody";  
  
  function handleClick() {  
    name = "John";  
  }  
  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```

No funciona!

home.tsx

```
import { useState } from "react";  
  
export default function Home() {  
  const [name, setName] =  
    useState("Anybody");  
  
  function handleClick() {  
    setName("John");  
  }  
  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```


Estado

La variable se inicializa con el
parámetro de useState(..)

home.tsx

```
export default function Home() {  
  let name = "Anybody";  
  function handleClick() {  
    name = "John";  
  }  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```

No funciona!

home.tsx

```
import { useState } from "react";  
export default function Home() {  
  const [name, setName] =  
    useState("Anybody");  
  function handleClick() {  
    setName("John");  
  }  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```

home.tsx

```
export default function Home() {  
  let name = "Anybody";  
  function handleClick() {  
    name = "John";  
  }  
  return (  
    <>  
      <h1>Hello {name}</h1>  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```

No funciona!

home.tsx

```
import { useState } from "react";  
export default function Home() {  
  const [name, setName] =  
    useState("Anybody");  
  function handleClick() {  
    setName("John");  
  }  
  return (  
    <>  
      <h1>Hello {name}!</h1>  
      <button onClick={handleClick}>  
        Hello John  
      </button>  
    </>  
  );  
}
```

La función se usa en vez
de asignar a la variable
un nuevo valor

- Un string como estado

home.tsx

```
import { useState } from "react";
export default function Home() {
  const [name, setName] = useState("Anybody");

  function handleClick() {
    setName("John");
  }

  return (
    <>
      <h1>Hello {name}!</h1>

      <button onClick={handleClick}>Hello John</button>
    </>
  );
}
```

- Un string como estado

home.tsx

```
import { useState } from "react";
export default function Home() {
  const [name, setName] = useState("Anybody");

  function handleClick() {
    setName("John");
  }

  return (
    <>
      <h1>Hello {name}!</h1>

      <button onClick={handleClick}>Hello John</button>
    </>
  );
}
```

useState devuelve un array con el valor (name) y una función para modificar el valor (setName)

Se configura el valor inicial

- Un string como estado

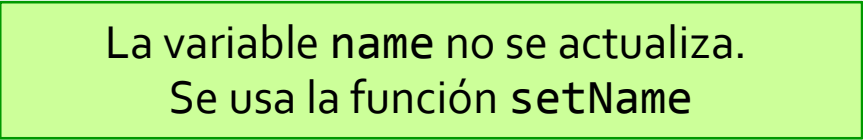
home.tsx

```
import { useState } from "react";
export default function Home() {
  const [name, setName] = useState("Anybody");

  function handleClick() {
    setName("John");
  }

  return (
    <>
      <h1>Hello {name}!</h1>

      <button onClick={handleClick}>Hello John</button>
    </>
  );
}
```



exam10_state_object

- **Un objeto como estado**

- Si el estado es un objeto y se **cambia el valor de un atributo, no es detectado** por React y el HTML no se actualiza
- Para que React lo **detecte**, hay que **reemplazar el objeto por otro** (aunque sea prácticamente igual, salvo un atributo)
- Esta estrategia hace muy **eficiente la detección de que el estado cambia** y hay que renderizar de nuevo el componente (generar HTML)

- Un objeto como estado

home.tsx

```
import { useState } from "react";

export default function Home() {

  const [user, setUser] = useState(
    { name: "John", age: 23 }
  );

  function handleClick() {
    //DON'T WORK: user.age++;
    setUser({ ...user, age: user.age + 1 });
  }

  return (
    <>
      <p>Name: {user.name}</p>
      <p>Age: {user.age}</p>
      <button onClick={handleClick}>Increment age</button>
    </>
  );
}
```

- Un objeto como estado

home.tsx

```
import { useState } from "react";

export default function Home() {

  const [user, setUser] = useState(
    { name: "John", age: 23 }
  );

  function handleClick() {
    //DON'T WORK: user.age++;
    setUser({ ...user, age: user.age + 1 });
  }

  return (
    <>
      <p>Name: {user.name}</p>
      <p>Age: {user.age}</p>
      <button onClick={handleClick}>Increment</button>
    </>
  );
}
```

La **mutación** del objeto no es detectada por React

El **destructuring** de JS simplifica la creación de un objeto similar a otro excepto por un atributo (**inmutabilidad**)

exam11_state_array

- **Un array como estado**

- Con los arrays ocurre lo mismo que con los objetos, o se **reemplaza el array completo** o sus cambios no se detectan
- Es necesario crear un **nuevo array** basado en el anterior

```
users.find(user => user.id === id)!.age++;
```



```
setUsers(  
  users.map(user => user.id === id ?  
    { ...user, age: user.age + 1 } : user)  
);
```



- Un array como estado

```
import { useState } from "react";

export default function Home() {

  const [users, setUsers] = useState([
    { id: 1, name: "John", age: 23 },
    { id: 2, name: "Peter", age: 45 }]
  );

  function incrementAge(id: number) {
    //DON'T WORK: users.find(user => user.id === id)!.age++;
    setUsers(
      users.map(user => user.id === id ?
        { ...user, age: user.age + 1 } : user));
  }

  return (
    users.map(user => <div key={user.id}>
      <p>Name: {user.name}</p>
      <p>Age: {user.age}</p>
      <button onClick={() => incrementAge(user.id)}>Increment age</button>
    </div>)
  );
}
```

- Un array como estado

```
import { useState } from "react";

export default function Home() {

  const [users, setUsers] = useState([
    { id: 1, name: "John", age: 23 },
    { id: 2, name: "Peter", age: 45 }
  ]);

  function incrementAge(id: number) {
    //DON'T WORK: users.find(user => user.id === id)!.age++;
    setUsers(
      users.map(user => user.id === id ?
        { ...user, age: user.age + 1 } : user));
  }

  return (
    users.map(user => <div key={user.id}>
      <p>Name: {user.name}</p>
      <p>Age: {user.age}</p>
      <button onClick={() => incrementAge(user.id)}>
    </div>)
  );
}
```

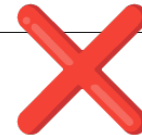
La **mutación** de un array no es detectada por React

Creamos un nuevo array con map(), manteniendo los objetos excepto el que cambia, que se crea nuevo basado en el anterior (**inmutabilidad**)

Estilos

- Las plantillas react son bastante similares al HTML, excepto algunos detalles:

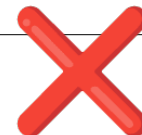
```
<h1 class="red">Title!</h1>
```



```
<h1 className="red">Title!</h1>
```



```
<p style="color: blue">Some text</p>
```



```
<p style={{ color: "blue" }}>Some text</p>
```



- Las plantillas react son bastante similares al HTML, excepto algunos detalles:

```
export default function Home() {  
  return (  
    <>  
      { /*DON'T WORK: <h1 class="red">Title!</h1>*/ }  
      <h1 className="red">Title!</h1>  
  
      { /*DON'T WORK: <p style="color: blue">Some text</p>*/ }  
      <p style={{ color: "blue" }}>Some text</p>  
    </>  
  );  
}
```

- El estado se puede aplicar al estilo (y puede cambiar)

```
import { useState } from "react";

export default function Home() {

  const [titleClass, setTitleClass] = useState("red");
  const [textStyle, setTextStyle] = useState({ color: "blue" });

  function handleClick() {
    setTitleClass("blue");
    setTextStyle({ color: "red" });
  }

  return (
    <>
      <h1 className={titleClass}>Title!</h1>
      <p style={textStyle}>Some text</p>

      <button onClick={handleClick}>Change Colors</button>
    </>
  );
}
```

Los estilos cambian



Formularios

- Acceso al Nodo DOM de los campos del formulario
 - Con un *hook* se define una variable de referencia

```
const inputRef = useRef<HTMLInputElement>(null);
```

- Un elemento HTML se vincula a la referencia

```
<input type="text" ref={inputRef}/>
```

- La propiedad **current** permite acceder al Nodo DOM (al procesar un evento)

```
let input = inputRef.current;  
setName(input?.value ?? "");  
input!.value = "";
```

- Acceso al Nodo DOM de los campos del formulario

```
import { useRef, useState } from "react";

export default function Home() {

  const [name, setName] = useState("Anybody");

  const inputRef = useRef<HTMLInputElement>(null);

  function handleClick() {
    let input = inputRef.current;
    setName(input?.value ?? "");
    input!.value = "";
  }

  return (
    <>
      <input type="text" ref={inputRef}/>

      <h1>Hello {name}!</h1>
      <button onClick={handleClick}>Update title</button>
    </>
  );
}
```

- Acceso al Nodo DOM de los campos del formulario

```
import { useRef, useState } from "react";

export default function Home() {

  const [name, setName] = useState("Anybody");

  const inputRef = useRef<HTMLInputElement>(null);

  function handleClick() {
    let input = inputRef.current;
    setName(input?.value ?? "");
    input!.value = "";
  }

  return (
    <>
      <input type="text" ref={inputRef}/>

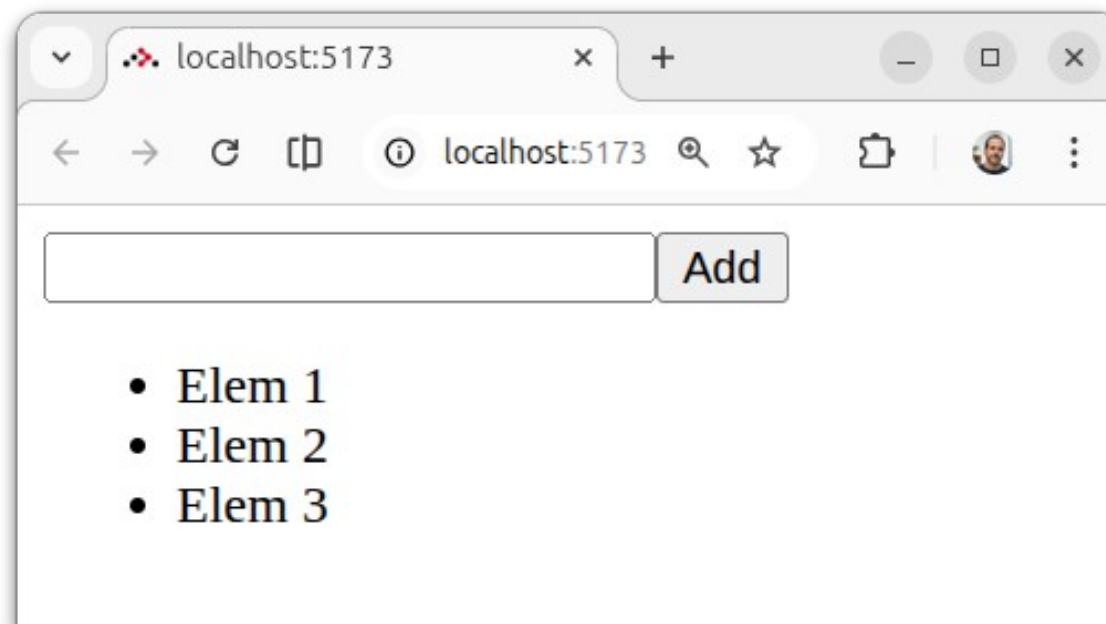
      <h1>Hello {name}!</h1>
      <button onClick={handleClick}>Update title</button>
    </>
  );
}
```

Se accede al
Nodo DOM

Se vincula el input a la
variable inputRef

Ejercicio 1

- Implementa una aplicación con un **campo de texto** y un **botón de Añadir**
- Cada vez que se pulse el **botón**, el **contenido** del campo de texto se **añadirá a la lista**



- Sincronizar un campo con un estado react
 - El valor del campo se lee del estado

```
<input type="text" value={name} .../>
```

- Se suscribe al evento onChange del input

```
<input type="text" value={name} onChange={onChange}/>
```

- Se actualiza el estado por cada cambio

```
function onChange(e: ChangeEvent<HTMLInputElement>) {  
  setName(e.target.value);  
}
```

- Sincronizar un campo con un estado react

```
import { useState, type ChangeEvent } from "react";

export default function Home() {

  const [name, setName] = useState("Anybody");

  function handleClick() {
    setName("John");
  }

  function onChange(e: ChangeEvent<HTMLInputElement>) {
    setName(e.target.value);
  }

  return (
    <>
      <input type="text" value={name} onChange={onChange}/>
      <h1>Hello {name}!</h1>
      <button onClick={handleClick}>Hello John</button >
    </>
  );
}
```

- Sincronizar un campo con un estado react

```
import { useState, type ChangeEvent } from "react";

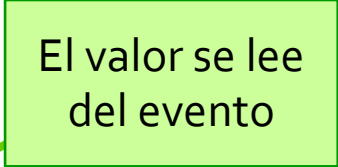
export default function Home() {

  const [name, setName] = useState("Anybody");

  function handleClick() {
    setName("John");
  }

  function onChange(e: ChangeEvent<HTMLInputElement>) {
    setName(e.target.value);
  }

  return (
    <>
      <input type="text" value={name} onChange={onChange}/>
      <h1>Hello {name}!</h1>
      <button onClick={handleClick}>Hello John</button >
    </>
  );
}
```



- Leer todos los campos de un formulario
 - Es habitual leer todos los campos del formulario al pulsar el botón para enviar (*submit*)
 - Se puede usar la clase FormData nativa del navegador

```
function handleSubmit(event: SubmitEvent) {  
    event.preventDefault();  
  
    const formData = new FormData(event.target);  
  
    let name = formData.get("name") as string;  
    let special = formData.get("special") === "on";  
    ...  
  
    event.target.reset();  
}
```

```
<form onSubmit={handleSubmit}>  
  
    <label htmlFor="name">Name:</label>  
    <input name="name" type="text" />  
  
    <label htmlFor="special">Special:</label>  
    <input name="special" type="checkbox" />  
  
    <button type="submit">Process</button>  
  
</form>
```


Lectura de todos los tipos de campos

```
import { useState, type SubmitEvent } from "react";

export default function Home() {

  interface User {
    name?: string;
    special?: boolean;
    gender?: string;
    country?: string;
  }

  const [user, setUser] = useState<User>();

  function handleSubmit(event: SubmitEvent) {

    event.preventDefault();

    const formData = new FormData(event.target);

    setUser({
      name: formData.get("name") as string,
      special: formData.get("special") === "on",
      gender: formData.get("gender") as string,
      country: formData.get("country") as string
    });

    event.target.reset();
  }
}
```

```
return (
  <>
    <form onSubmit={handleSubmit}>

      <h1>Create user</h1>

      <label htmlFor="name">Name: </label>
      <input name="name" type="text" />
      <br />

      <label htmlFor="special">Special: </label>
      <input name="special" type="checkbox" />
      <br />

      <label htmlFor="gender">Gender: </label>
      <input name="gender" value="male" type="radio" /> Male
      <input name="gender" value="female" type="radio" /> Female
      <br />

      <label htmlFor="country">Country: </label>
      <select name="country">
        <option value="" disabled selected hidden>Select...</option>
        <option value="spain">Spain</option>
        <option value="usa">USA</option>
        <option value="france">France</option>
      </select>
      <br />

      <button type="submit">Process form</button >
    </form>

    {user && (
      <p> Name: {user.name ? user.name : "Not specified"}<br />
        Special: {user.special ? "Yes" : "No"}<br />
        Gender: {user.gender ? user.gender : "Not specified"}<br />
        Country: {user.country ? user.country : "Not specified"}
      </p >)
    }
  </>
);
}
```

- Se pueden sincronizar todos los campos con las propiedades de un objeto

```
interface User {  
  name?: string;  
  special?: boolean;  
  gender?: string;  
  country?: string;  
}  
  
const [user, setUser] = useState<User>();
```

Input texto

```
function handleChangeName(event: ChangeEvent<HTMLInputElement>) {  
  setUser({ ...user, name: event.target.value });  
}
```

```
<label htmlFor="name">Name: </label>  
<input name="name" value={user?.name ?? ""} type="text" onChange={handleChangeName} />
```

- Se pueden sincronizar todos los campos con las propiedades de un objeto

Checkbox

```
function handleChangeSpecial(event: ChangeEvent<HTMLInputElement>) {  
  setUser({ ...user, special: event.target.checked });  
}
```

```
<label htmlFor="special">Special: </label>  
<input name="special" checked={user?.special ?? false}  
  type="checkbox" onChange={handleChangeSpecial} />
```

- Se pueden sincronizar todos los campos con las propiedades de un objeto

Radio Buttons

```
function handleChangeGender(event: ChangeEvent<HTMLInputElement>) {  
  setUser({ ...user, gender: event.target.value });  
}
```

```
<label htmlFor="gender">Gender: </label>  
  
<input name="gender" value="male" checked={user?.gender === "male"}  
  type="radio" onChange={handleChangeGender} /> Male  
  
<input name="gender" value="female" checked={user?.gender === "female"}  
  type="radio" onChange={handleChangeGender} /> Female
```

- Se pueden sincronizar todos los campos con las propiedades de un objeto

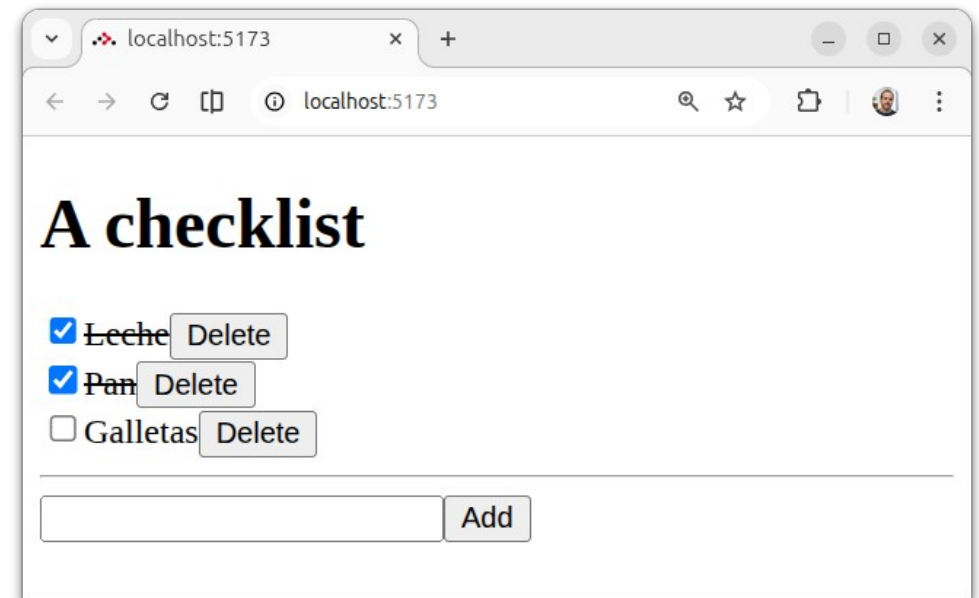
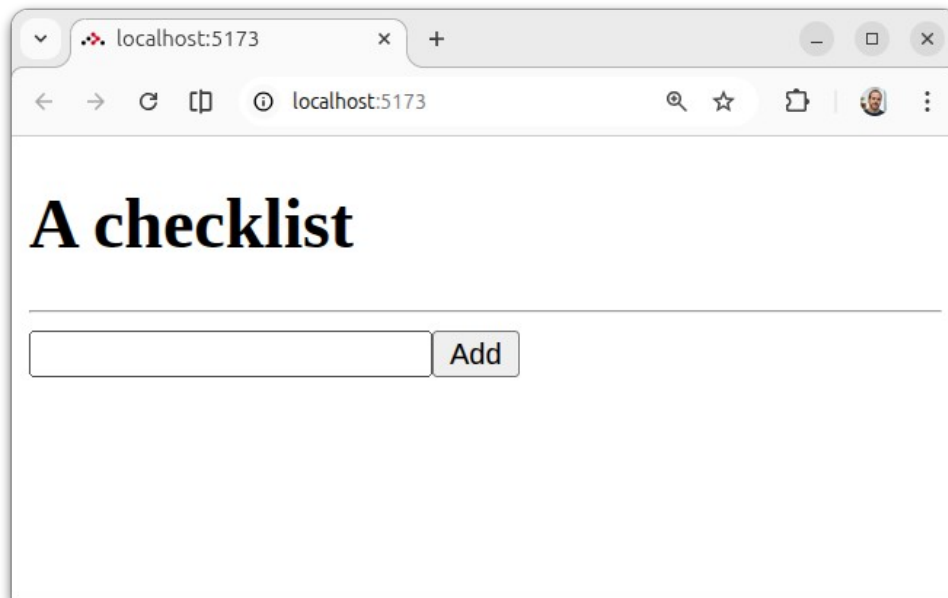
Select

```
function handleChangeCountry(event: ChangeEvent<HTMLSelectElement>) {  
    setUser({ ...user, country: event.target.value });  
}
```

```
<label htmlFor="country">Country: </label>  
<select name="country" onChange={handleChangeCountry}>  
    <option value="" disabled selected hidden>Select...</option>  
    <option value="spain" selected={user?.country === "spain"}>Spain</option>  
    <option value="usa" selected={user?.country === "usa"}>USA</option>  
    <option value="france" selected={user?.country === "france"}>France</option>  
</select>
```

Ejercicio 2

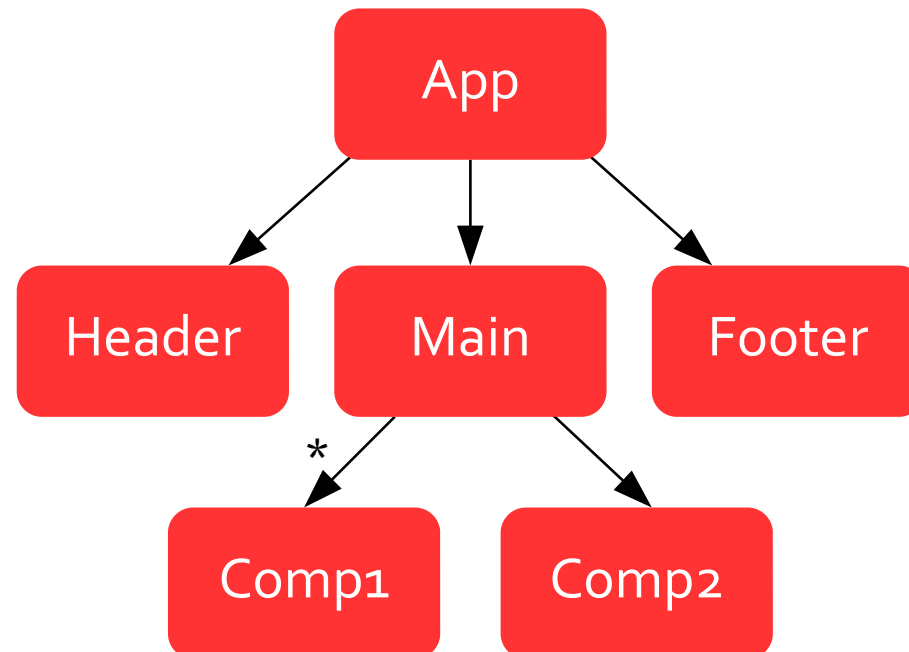
- Implementa una aplicación de **gestión de elementos**
- Los elementos se mantendrán en **memoria**



Composición de Componentes

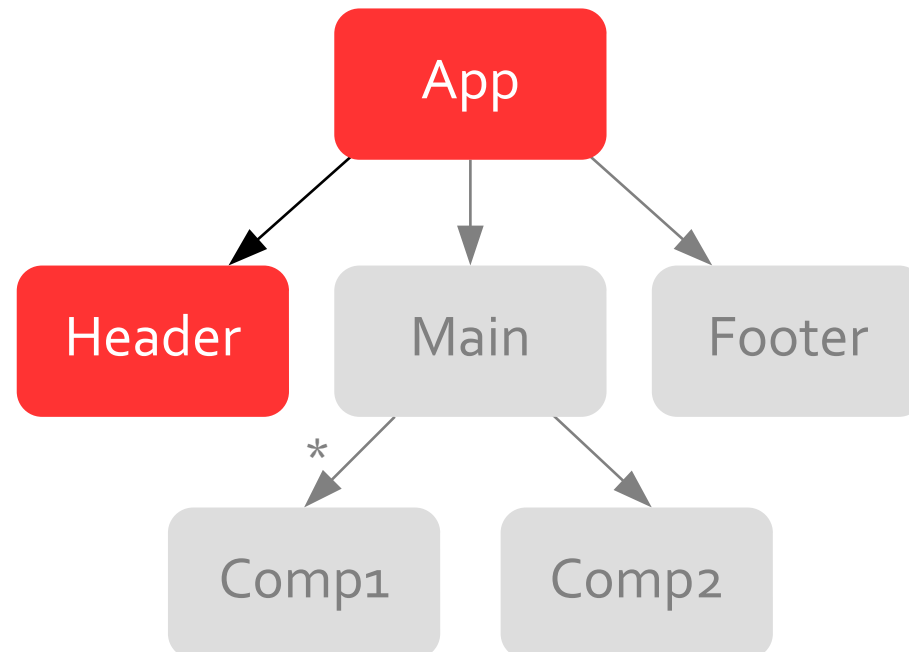
Árboles de componentes

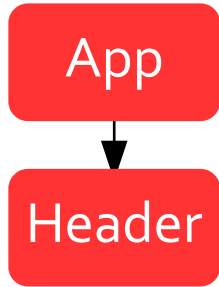
En React un componente puede estar formado por más componentes formando un árbol



Árboles de componentes

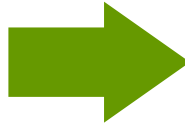
En React un componente puede estar formado por más componentes formando un árbol





Árboles de componentes

```
<h1>Title</h1>
<p>Main content</p>
```



```
<Header/>
<p>Main content</p>
```

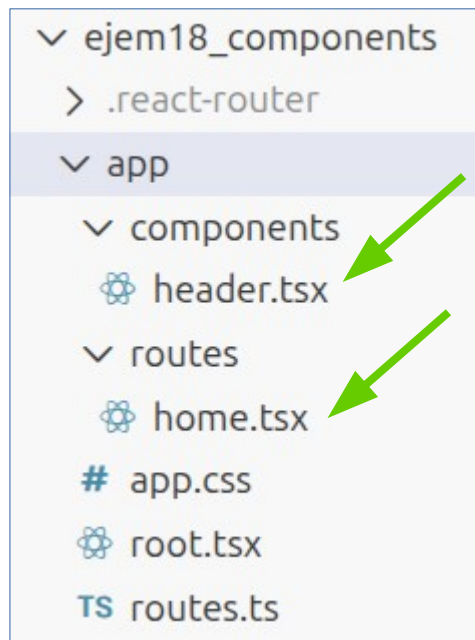
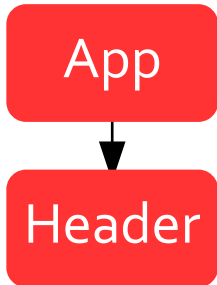
```
<Header>
```

```
<h1>Title</h1>
```

Composición de componentes

exam18_components

Árboles de componentes



home.tsx

```
import Header from "~/components/header";

export default function Home() {

  return (
    <>
      <Header />
      <p>Main content</p>
    </>
  );
}
```

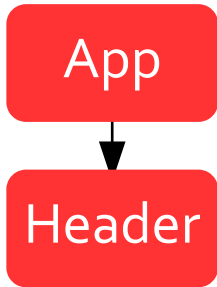
header.tsx

```
export default function Header() {

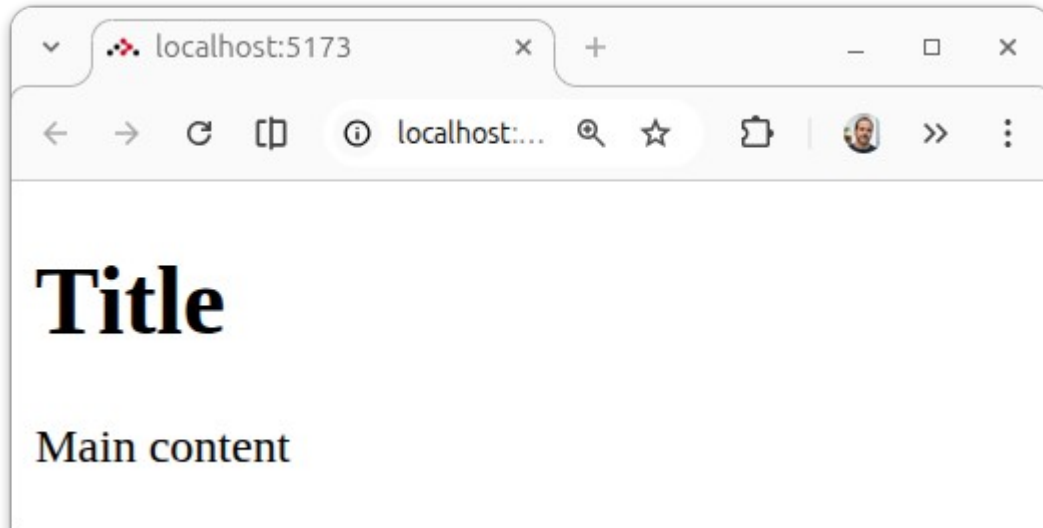
  return <h1>Title</h1>;
}
```

Composición de componentes

exam18_components



Árboles de componentes



```
<!DOCTYPE html>
<html lang="en">
  > <head> ... </head>
  ▼ <body>
    <h1>Title</h1>
    <p>Main content</p>
```

Composición de componentes

- Comunicación entre un **componente padre** y un **componente hijo**
 - Propiedades
 - Contexto
 - Store

- El componente padre puede especificar **propiedades** (Props) en el componente hijo

exam19_props

home.tsx

```
import Header from "~/components/header";

export default function Home() {

  const title = "Welcome to Home";

  return (
    <>
      <Header title={title} />
      <p>Main content</p>
    </>
  );}
```

header.tsx

```
interface HeaderProps { title: string; }

export default function Header({ title }: HeaderProps) {

  return <h1>{title}</h1>;
}
```

Propiedades

- El componente padre puede especificar **propiedades** (Props) en el componente hijo

exam19_props

home.tsx

```
import Header from "~/components/header";

export default function Home() {

  const title = "Welcome to Home";

  return (
    <>
      <Header title={title} />
      <p>Main content</p>
    </>
  );}
```

Se le pasa la propiedad **title**

Se define de tipo **string**

El title se recibe como una propiedad del objeto HeaderProps (se extrae con **destructuring**)

header.tsx

```
interface HeaderProps { title: string; }

export default function Header({ title }: HeaderProps) {

  return <h1>{title}</h1>;
}
```

- El componente hijo puede generar eventos si recibe una **función** del componente padre en las props

home.tsx

```
import Header from "~/components/header";

export default function Home() {

  const title = "Welcome to Home";

  function handleChange(visible: boolean) {
    console.log("Header visible: " + visible);
  }

  return (
    <>
      <Header title={title} onChange={handleChange} />
      <p>Main content</p>
    </>
  );
}
```

Se pasa la función
como a un elemento
nativo HTML



Propiedades

header.tsx

exam20_props_events

```
import { useState } from "react";

interface HeaderProps {
  title: string;
  onChange: (visible: boolean) => void;
}

export default function Header({ title, onChange }: HeaderProps) {

  const [visible, setVisible] = useState(true);

  function handleClick() {
    const newVisible = !visible;
    setVisible(newVisible);
    if(onChange) {
      onChange(newVisible);
    }
  }

  return (
    <>
      {visible} && <h1>{title}</h1>
      <button onClick={handleClick}>Hide/Show</button>
    </>
  );
}
```

Un prop es un manejador de eventos (que puede tener parámetros)

Si se ha configurado el manejador de eventos, es invocado cuando se produce el evento

- Se puede pasar HTML a un componente

home.tsx

```
import Header from "~/components/header";

export default function Home() {

  return (
    <>
      <Header>
        <h1>Welcome to Home</h1>
      </Header>
      <p>Main content</p>
    </>
  );}
```

header.tsx

```
import { useState, type ReactNode } from "react";

interface HeaderProps {
  children: ReactNode;
}

export default function Header({ children }: HeaderProps) {

  return (<div>{children}</div>);
}
```

Propiedades

- Se puede pasar HTML a un componente

home.tsx

```
import Header from "~/components/header";

export default function Home() {

  return (
    <>
      <Header>
        <h1>Welcome to Home</h1>
      </Header>
      <p>Main content</p>
    </>
  );}
```

El HTML se pasa como elemento hijo (a nivel DOM)

header.tsx

```
import { useState, type ReactNode } from "react";

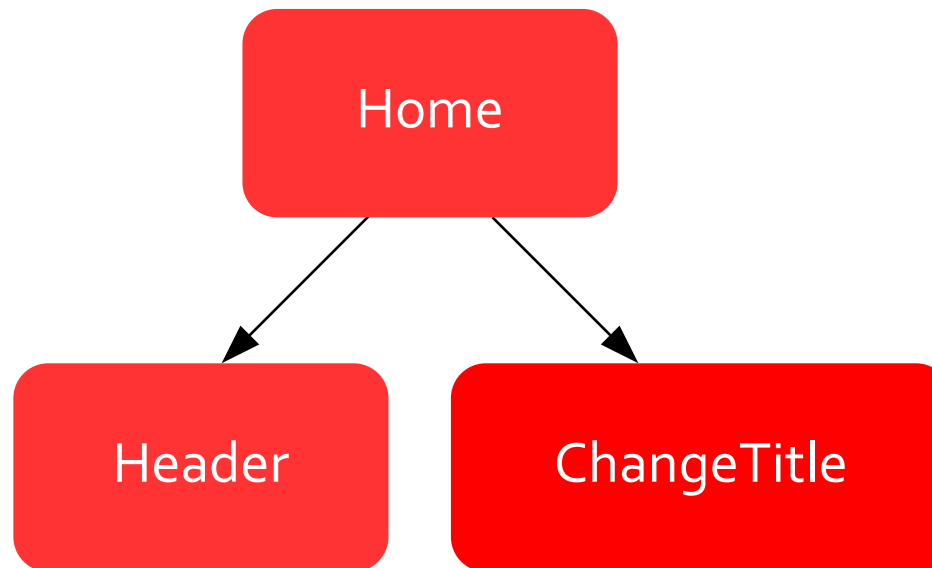
interface HeaderProps {
  children: ReactNode;
}

export default function Header({ children }: HeaderProps) {

  return (<div>{children}</div>);
}
```

Se recibe como una prop de tipo ReactNode

- Dos componentes se pueden sincronizar si el padre les pasa el mismo estado



- Dos componentes se pueden sincronizar si el padre les pasa el mismo estado

home.tsx

```
export default function Home() {  
  
  const [title, setTitle] = useState("Welcome to Home");  
  
  return (  
    <>  
      <Header title={title} />  
      <p>Main content</p>  
      <ChangeTitle setTitle={setTitle} />  
    </>  
  );  
}
```

change-title.tsx

```
interface ChangeTitleProps {  
  setTitle: (title: string) => void;  
}  
  
export default  
function ChangeTitle({ setTitle }: ChangeTitleProps){  
  
  return (  
    <button onClick={() => setTitle("New Title")}>  
      Set New Title  
    </button>  
  );  
}
```

header.tsx

```
interface HeaderProps {  
  title: string;  
}  
  
export default  
function Header({ title }: HeaderProps){  
  
  return (<h1>{title}</h1>);  
}
```

Propiedades

exam22_sync_components

- Dos componentes se pueden sincronizar si el padre les pasa el mismo estado

home.tsx

```
export default function Home() {  
  const [title, setTitle] = useState("Welcome to Home");  
  
  return (  
    <>  
      <Header title={title} />  
      <p>Main content</p>  
      <ChangeTitle setTitle={setTitle} />  
    </>  
  );  
}
```

Cada hijo recibe lo que necesita

change-title.tsx

```
interface ChangeTitleProps {  
  setTitle: (title: string) => void;  
}  
  
export default  
function ChangeTitle({ setTitle }: ChangeTitleProps){  
  return (  
    <button onClick={() => setTitle("New Title")}>  
      Set New Title  
    </button>  
  );  
}
```

Se cambia el valor al pulsar el botón

header.tsx

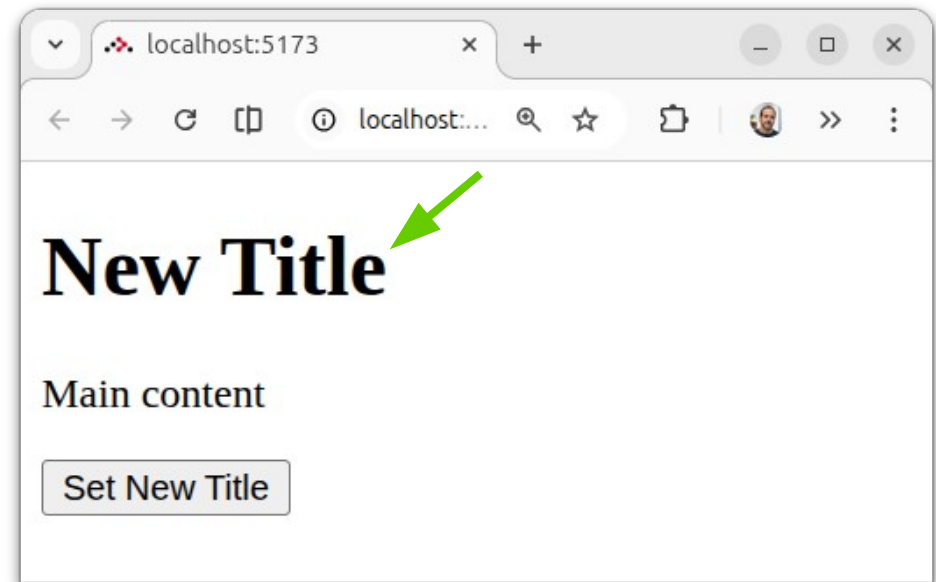
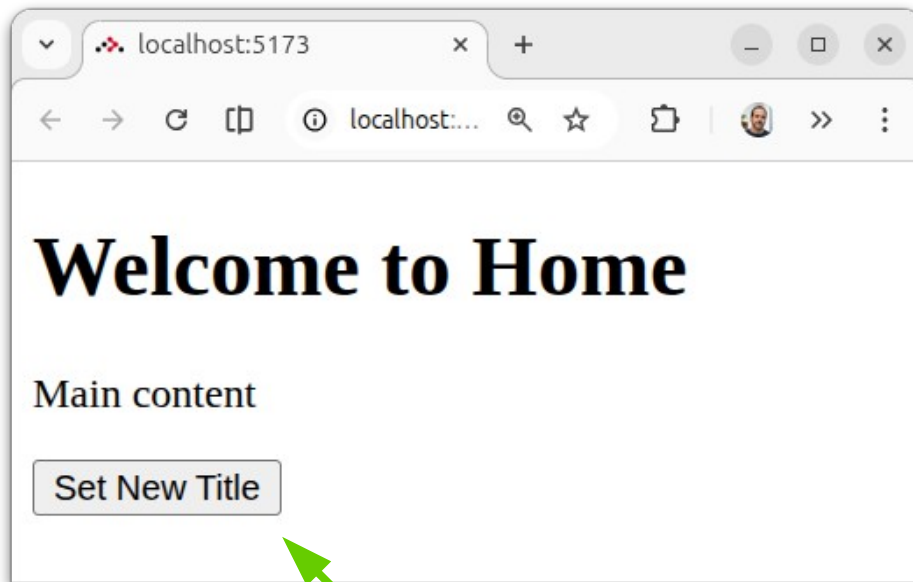
```
interface HeaderProps {  
  title: string;  
}  
  
export default  
function Header({ title }: HeaderProps){  
  return (<h1>{title}</h1>);  
}
```

El valor se actualiza

Propiedades

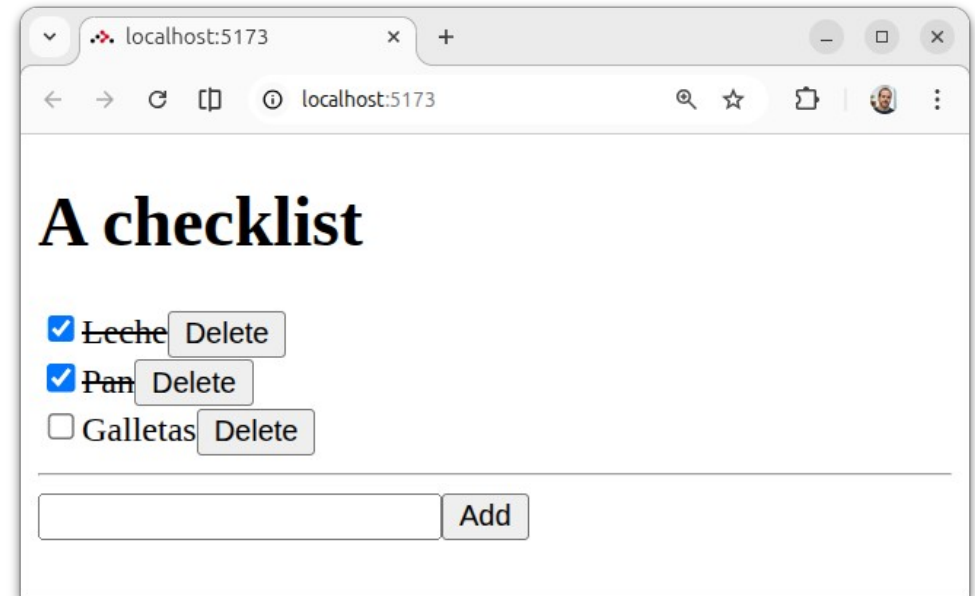
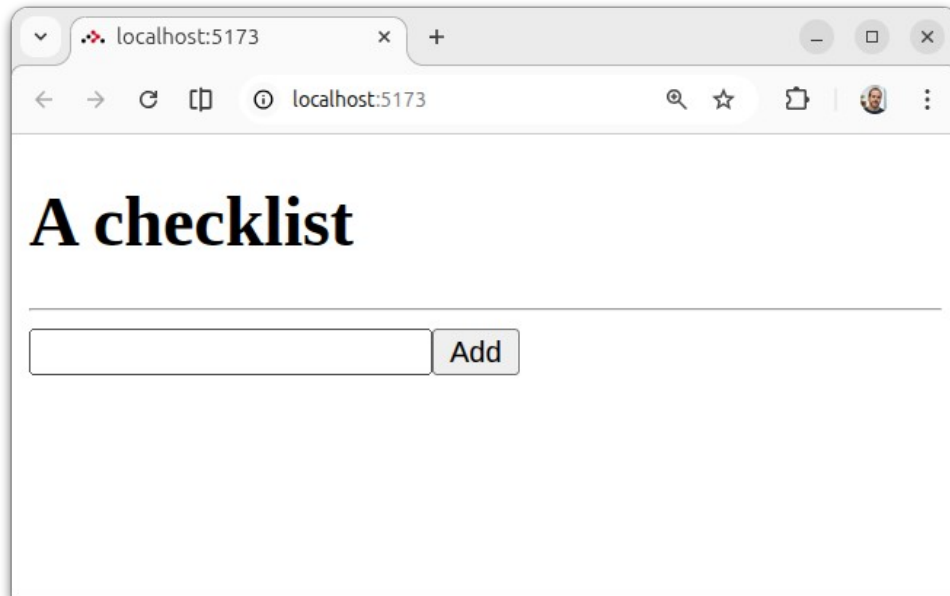
exam22_sync_components

- Dos componentes se pueden sincronizar si el padre les pasa el mismo estado



Ejercicio 3

- Refactoriza la aplicación de **gestión de elementos** para que el componente `ItemRow` visualice una única tarea



Estado compartido

- Un **contexto** se utiliza para compartir información entre todos los componentes de una jerarquía sin pasarla como propiedades nivel a nivel
- Se utiliza principalmente para que un componente padre pueda pasar “contexto” a componentes **alejados** en la jerarquía (**nietos**, biznietos...)
- Implementación
 - Se **define un contexto** (nombre y estructura)
 - El componente padre **engloba a los componentes** que tendrán acceso al contexto en la plantilla (componente invisible)
 - Los descendientes **acceden al contexto**

- Contexto
 - Estructura de datos y operaciones
 - Opcionalmente el valor inicial

contexts/title-context.tsx

```
import { createContext } from "react";

interface TitleContextType {
  title: string;
  setTitle: (title: string) => void;
}

export const TitleContext = createContext<TitleContextType>(null!);
```

- Contexto
 - Estructura de datos y operaciones
 - Opcionalmente el valor inicial

contexts/title-context.tsx

```
import { createContext } from "react";  
  
interface TitleContextType {  
  title: string;  
  setTitle: (title: string) => void;  
}  
  
export const TitleContext = createContext<TitleContextType>(null!);
```

Estructura

Valor inicial

home.tsx

```
...
export default function Home() {

  const [title, setTitle] = useState("Welcome to Home");

  return (
    <TitleContext.Provider value={{ title, setTitle }}>
      <Header />
      <p>Main content</p>
      <ChangeTitle />
    </TitleContext.Provider>
  );
}
```

change-title.tsx

```
...
export default function ChangeTitle() {

  const { setTitle } = useContext(TitleContext);

  return (
    <button onClick={() => setTitle("New Title")}>
      Set New Title
    </button>
  );
}
```

header.tsx

```
...
export default function Header() {

  const { title } = useContext(TitleContext);

  return (
    <h1>{title}</h1>
  );
}
```

Contexto

exam23_context

home.tsx

```
...  
export default function Home() {  
  const [title, setTitle] = useState("Welcome to Home");  
  return (  
    <TitleContext.Provider value={{ title, setTitle }}>  
      <Header />  
      <p>Main content</p>  
      <ChangeTitle />  
    </TitleContext.Provider>  
  );  
}
```

Engloba a los
componentes que
comparten
contexto

Estado del
contexto

change-title.tsx

```
...  
export default function ChangeTitle() {  
  const { setTitle } = useContext(TitleContext);  
  return (  
    <button onClick={() => setTitle("New Title")}>  
      Set New Title  
    </button>  
  );  
}
```

header.tsx

```
...  
export default function Header() {  
  const { title } = useContext(TitleContext);  
  return (  
    <h1>{title}</h1>  
  );  
}
```

Cada componente puede
acceder al contexto con el hook
useContext(...)

exam24_context_state

- Se puede crear un componente para **gestionar el contexto (XXProvider)**, pero “invisible”
- Declara estado y operaciones
- La plantilla sólo incrusta el children que recibe como parámetro

- Definición de contexto

```
import { createContext, useState, type ReactNode } from "react";

interface TitleContextType {
  title: string;
  toUpperCase: () => void;
}

export const TitleContext = createContext<TitleContextType>(null!);

export function TitleProvider({ children }: { children: ReactNode }) {
  const [title, setTitle] = useState("Welcome to Home");

  function toUpperCase() {
    setTitle(title.toUpperCase());
  }

  return (
    <TitleContext.Provider value={{ title, toUpperCase }}>
      {children}
    </TitleContext.Provider>
  );
}
```

Componente

Estructura

Valor inicial

Estado

Operaciones

Plantilla
"invisible"

home.tsx

```
...  
export default function Home() {  
  
  const [title, setTitle] = useState("Welcome to Home");  
  
  return (  
    <TitleContext.Provider value={{ title, setTitle }}>  
      <Header />  
      <p>Main content</p>  
      <ChangeTitle />  
    </TitleContext.Provider>  
  );  
}
```



home.tsx

```
...  
export default function Home() {  
  
  return (  
    <TitleProvider>  
      <Header />  
      <p>Main content</p>  
      <ChangeTitle />  
    </TitleProvider>  
  );  
}
```

El Home ya
no gestiona el
estado

exam24_context_state

home.tsx

```
...  
export default function Home() {  
  
  return (  
    <TitleProvider>  
      <Header />  
      <p>Main content</p>  
      <ChangeTitle />  
    </TitleProvider>  
  );  
}
```

change-title.tsx

```
...  
export default function ChangeTitle() {  
  
  const { setTitle } = useContext(TitleContext);  
  
  return (  
    <button onClick={() => setTitle("New Title")}>  
      Set New Title  
    </button>  
  );  
}
```

header.tsx

```
...  
export default function Header() {  
  
  const { title } = useContext(TitleContext);  
  
  return (  
    <h1>{title}</h1>  
  );  
}
```

El uso no cambia

- El contexto de React requiere declararse como un componente en la plantilla
- Si hay varios contextos tienen que anidarse

```
...  
  
export default function Home() {  
  return (  
    <TitleContext.Provider ...>  
      <OtherContext.Provider ...>  
        <Header />  
        <p>Main content</p>  
        <ChangeTitle />  
      </OtherContext.Provider>  
    </TitleContext.Provider>  
  );  
}
```

Ejercicio 4

- Refactoriza la aplicación de **gestión de elementos** para que la lista de tareas se gestione en un contexto
- Crea el componente `ItemsManager` para desacoplar de la aplicación principal (`home.tsx`)

- Zustand es una librería externa

```
npm install zustand
```

- Simplifica la gestión de estado compartido en un ***store*** (tienda) entre componentes
- Su implementación es **más sencilla** que el contexto y no se especifica en la plantilla (evitando el **anidamiento**)



<https://zustand.docs.pmnd.rs/>

- Clase store

contexts/title-store.tsx

```
import { create } from 'zustand';

interface TitleStore {
  title: string;
  setTitle: (title: string) => void;
}

export const useTitleStore = create<TitleStore>((set) => ({
  title: 'Welcome to Home',
  setTitle: (title) => set({ title }),
}));
```

- Clase store

contexts/title-store.tsx

```
import { create } from 'zustand';

interface TitleStore {
  title: string;
  setTitle: (title: string) => void;
}

export const useTitleStore = create<TitleStore>((set) => ({
  title: 'Welcome to Home',
  setTitle: (title) => set({ title }),
}));
```

Interfaz

Implementación

Store

exam25_store

home.tsx

```
import ChangeTitle from "~/components/change-title";
import Header from "~/components/header";

export default function Home() {
  return (
    <>
      <Header />
      <p>Main content</p>
      <ChangeTitle />
    </>
  );
}
```

header.tsx

```
import { useTitleStore } from "~/stores/title-store";

export default function Header() {
  const { title } = useTitleStore();

  return (
    <h1>{title}</h1>
  );
}
```

change-title.tsx

```
import { useTitleStore } from "~/stores/title-store";

export default function ChangeTitle() {
  const { setTitle } = useTitleStore();

  return (
    <button onClick={() => setTitle("New Title")}>
      Set New Title
    </button>
  );
}
```


Store

exam25_store

home.tsx

```
import ChangeTitle from "~/components/change-title";
import Header from "~/components/header";

export default function Home() {
  return (
    <>
      <Header />
      <p>Main content</p>
      <ChangeTitle />
    </>
  );
}
```

No se requiere
especificar el store

change-title.tsx

```
import { useTitleStore } from "~/stores/title-store";

export default function ChangeTitle() {

  const { setTitle } = useTitleStore();

  return (
    <button onClick={() => setTitle("New Title")}>
      Set New Title
    </button>
  );
}
```

header.tsx

```
import { useTitleStore } from "~/stores/title-store";

export default function Header() {

  const { title } = useTitleStore();

  return (
    <h1>{title}</h1>
  );
}
```

Acceso al store

- **Local vs Global**

- Un contexto es **local** al árbol de componentes en el que se define:
 - Podría haber contextos diferentes con **estados diferentes** sólo accesibles por parte de los componentes
- Un store es **global** a la aplicación
 - Todos los componentes acceden al **mismo estado**
- Existen formas de hacer que los stores sean locales, pero no son sencillas

- **Optimizaciones**

- Actualmente, cuando cambia el estado en un store, se actualizan todos los componentes que lo usan
- Se puede indicar qué parte del estado se usa de un store (selectores) para que los componentes sólo se rendericen de nuevo si el cambio les afecta

<https://zustand.docs.pmnd.rs/learn/guides/prevent-rerenders-with-use-shallow>

Ejercicio 5

- Refactoriza la aplicación de **gestión de elementos** para que la lista de tareas se gestione con un store de Zustand en vez de con un contexto

Composición de componentes

- ¿Cuándo crear un nuevo componente?
 - El ejercicio y los ejemplos son excesivamente **sencillos** para que compense la creación de un nuevo componente hijo
 - En casos reales se crearían nuevos componentes:
 - Cuando la lógica y/o la plantilla sean suficientemente **complejos**
 - Cuando los componentes hijos puedan **reutilizarse** en varios contextos

Composición de componentes

- ¿Cuándo usar props, contexto o store?
 - **Props:** Cuando la comunicación es directa entre padre e hijo
 - **Context:** Cuando el estado es local y la comunicación puede ir hasta nietos, biznietos...
 - **Store:** Cuando el estado es global